# Otonom Sistemlerde Ubuntu, Gömülü Sistem Platformları ve C++ Otonomi Algoritmaları Araştırması

Araştırma Konuları:


*1. Ubuntu İşletim Sistemi*

1.1. Ubuntu Nedir?
1.2. Otonom Sistemlerde Ubuntu'nun Kullanımı
Linux
Terminal
Paket Yöneticisi
Kütüphane ve Bağımlılıklar
ROS2

*2. NVIDIA Jetson ve Ubuntu*
2.1. NVIDIA Jetson Nedir?
2.2. Yazılım Ortamı
2.3. Otonom Sistemlerde Kullanımı

*3. Raspberry Pi ve Ubuntu*
3.1. Raspberry Pi Nedir?
3.2. Yazılım ortamı
3.3 Otonom Sistemlerde Kullanımı

*4. C++ ve Otonomi Algoritmaları*
4.1. C++ Dilinin Otonom Sistemlerdeki Rolü
4.2. Temel Otonomi Algoritmaları
Durum Makineleri( State Machine)
Yol Planlama (Path Planning)
Konum ve Yönelim Kestirimi
Engel Algılama ve Engelden Kaçınma
Kontrol Algoritmaları
4.3. ROS 2 ve C++
Örnek Uygulama

*5. Genel Değerlendirme*
*7. Sonuç*
*9. Kaynakça*



Hazırlayan: Meryem Akın


Tarih: 13 Ekim 2026

# 1. Ubuntu İşletim Sistemi

## *1.1. Ubuntu Nedir?*

Ubuntu, Linux tabanlı özgür ve ücretsiz bir işletim sistemidir. Bilgisayarlar, sunucular ve akıllı telefonlara yönelik olarak geliştirilmektedir. Ubuntu projesi Linux ve özgür yazılımın, bilgisayar kullanıcılarının günlük yaşamının bir parçası haline gelmesi amacıyla başlatılmış olup ilk kararlı masaüstü sürümü Ekim 2004'te yayınlanmıştır. Ubuntu'nun masaüstü sürümü günümüzde 40 milyonu aşkın kullanıcı sayısıyla dünyanın en yaygın kullanılan masaüstü Linux dağıtımı konumundadır.

Ubuntu, herkesin özgürce kullanabildiği; yayınlamakta, kopyalamakta ve kodlarını değiştirip geliştirebilmekte özgür olduğu yazılımlardan oluşur. Bunların çoğu GNU Genel Kamu Lisansı ile güvence altına alınmış, özgür yazılım / açık kaynak yazılımlardır.

Ubuntu Güney Afrikalı girişimci Mark Shuttleworth ve şirketi Canonical Ltd. tarafından finanse edilmektedir. Canonical, Ubuntu'yu bireysel ve kurumsal tüm kullanıcılara tamamen ücretsiz olarak sunmakta, teknik destek almak isteyen kuruluşlara destek vererek gelir elde etmeyi amaçlamaktadır. Canonical Ubuntu'yu açık kaynak kodlu ve özgür yazılım olarak sunduğu için, dünya çapında bu işletim sistemini kullanan ve geliştiren gönüllü kullanıcıları sayesinde, tüm bir işletim sistemini tek başına geliştirmek zorunda kalmaz. Diğer Linux katkıcılarının yaptığı geliştirmeler, onu temel alan Ubuntu'yu da doğrudan geliştirir.

Ubuntu sözcüğü, Zulu dilinde "insanlık" anlamına gelir, aynı zamanda "başkalarına karşı merhametli, şefkatli, iyiliksever" olmak gibi insani değerlerin temel alındığı bir dünya görüşüdür. 12 Buradan hareketle Ubuntu, İnsanlık için Linux (Linux For Human Beings) sloganını kullanır.

## *1.2. Otonom Sistemlerde Ubuntu'nun Kullanımı*

Otonom sistemlerde ve modern robotik mimarilerde Ubuntu, sunduğu kararlılık, geniş donanım desteği ve açık kaynak ekosistemi sayesinde endüstri standardı olarak kabul edilmektedir. Özellikle Robot İşletim Sistemi 2 (ROS 2) ile kusursuz bir uyum içinde çalışan Ubuntu, LIDAR, kamera ve IMU gibi yüksek bant genişliğine sahip sensör verilerinin anlık olarak işlenmesinde, filtrelenmesinde

ve merkezi olmayan düğüm (node) yapılarında kritik bir rol oynar. ROS 2 sürümlerinin (örneğin Humble veya Jazzy sürümleri) doğrudan belirli Ubuntu sürümleri (Ubuntu 22.04, 24.04 vb.) hedef alınarak optimize edilmesi, geliştiricilere donanım sürücülerinde ve DDS (Veri Dağıtım Hizmeti) altyapısında minimum hata ile kararlı bir geliştirme ortamı sunar. Bu güçlü entegrasyon; otonom araçların ve mobil robotların çevresel algılama, haritalama (SLAM) ve anlık yörünge planlama gibi yoğun veri işleme gerektiren görevleri güvenle yerine getirmesini sağlar.

## Linux

Linux çekirdeğine dayalı, açık kaynak kodlu, Unix benzeri bir işletim sistemi ailesidir. GNU Genel Kamu Lisansı versiyon 2 ile sunulan ve Linux Vakfıçatısı altında geliştirilen bir özgür yazılım projesidir. Linux ismi ilk geliştiricisi olan Linus Torvalds tarafından 1991 yılında verilmiştir. Günümüzde süper bilgisayarlarda, akıllı cihazların ve internet altyapısında kullanılan cihazların işletim sistemlerinde yaygın olarak kullanılmaktadır. Bunlardan en popüler olanı Google tarafından geliştirilen Android işletim sistemidir.

## Terminal

Terminal, bilgisayarlar ve işletim sistemleri ile etkileşimde bulunmanızı ve komutları, işlemleri veya görevleri yürütmenizi sağlayan bir kullanıcı arabirimi veya uygulamadır. Ayrıca “komut istemi,” “konsol,” veya “shell” olarak da adlandırılır. Bir terminal, kullanıcıların bilgisayarın işletim sistemine doğrudan erişmelerine ve komutları yazmalarına olanak tanır.
Terminaller, özellikle Unix ve Unix benzeri işletim sistemlerinde yaygın olarak kullanılır, ancak diğer işletim sistemlerinde de bulunabilirler. İşte terminallerin bazı temel işlevleri:

*Komut Girme:* Kullanıcılar, terminal penceresine komutlar veya komut satırı komutları girerek belirli görevleri gerçekleştirebilirler. Örneğin, dosya yönetimi, uygulama başlatma veya ağ araştırması gibi görevleri yürütebilirler.

*Dosya ve Klasör Yönetimi:* Terminal, dosya ve klasörleri görüntülemek, oluşturmak, silmek, taşımak ve düzenlemek için kullanılabilir. Komut satırı komutları, bu işlemleri yapmak için kullanılır.

*Sistem Yönetimi:* İşletim sistemi düzeyinde, terminal, sistem ayarlarını yapılandırmak, hizmetleri başlatmak veya durdurmak, güncellemeleri yönetmek ve kullanıcı hesaplarını yönetmek gibi yönetimsel görevleri gerçekleştirmek için kullanılır.

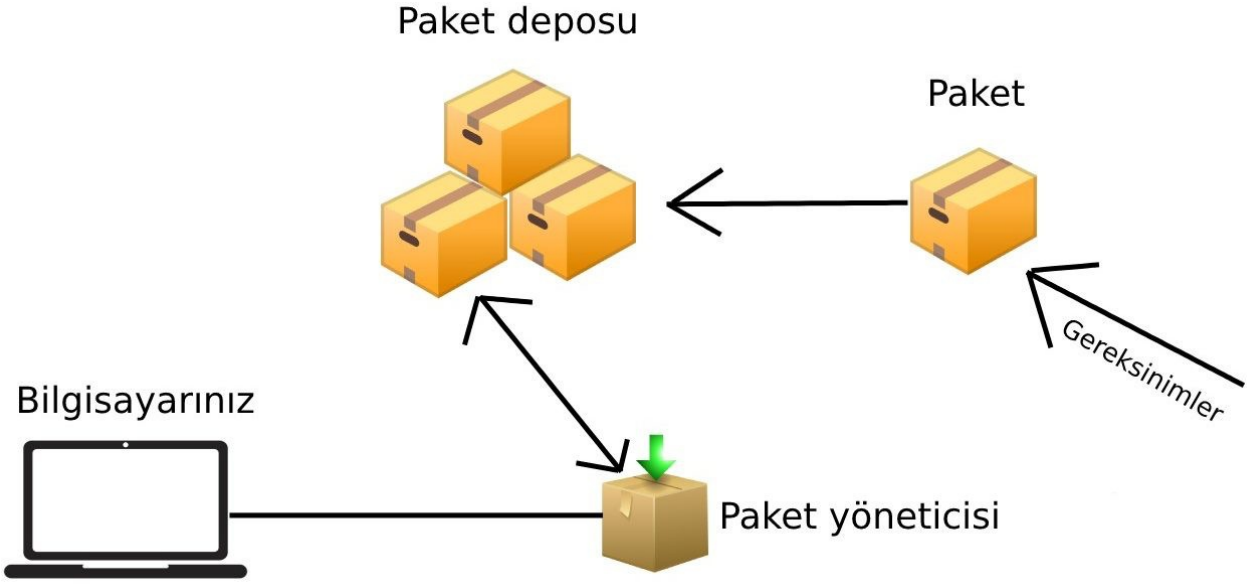
*Ağ Yönetimi:* Terminal, ağ bağlantılarını yönetmek, IP adreslerini yapılandırmak, ağ cihazlarını taramak ve ağ hizmetlerini izlemek gibi ağ yönetimi görevleri için kullanılabilir.

*Hata Ayıklama:* Yazılım geliştirme süreçlerinde, geliştiriciler hata ayıklama (debugging) için terminali kullanabilirler. Hata mesajlarını görüntülemek, kodu test etmek ve hata tespiti yapmak için kullanılır.

*Scripting ve Otomasyon:* Terminal, komut satırı betikleri (script) oluşturmak ve otomasyon görevleri yürütmek için kullanılabilir. Bu, belirli işlemleri otomatikleştirmek için faydalıdır.
Terminaller, aynı zamanda işletim sistemlerinin daha derin seviyelerine erişim sağlar ve gelişmiş kullanıcılar için güçlü bir araç olabilirler. Bununla birlikte, terminallerin kullanımı biraz teknik beceri gerektirebilir ve dikkatli kullanım gerektirebilir, çünkü yanlış bir komut sistemin çalışmasını etkileyebilir veya veri kaybına neden olabilir.

## Paket Yöneticisi

Paket yöneticilerinin asıl amacı, yazılımları güvenli ve pratik bir şekilde kontrol etmektir. Bu yazılımlar sayesinde, dağıtım sağlayıcınızın yazılım deposunda aradığınızı bulabilirsiniz. Örneğin Debian dağıtımı kullanıcılarının APT yazılımı, Debian'ın yazılım depolarına bağlıdır. Çoğu paket yönetim yazılımı üçüncü parti depoların eklenmesine de izin verir. Arch Linux dağıtımında, üçüncü parti kullanıcıların yazılım yükleyebilecekleri bir depo bulunmaktadır.
Paket yönetim yazılımları, grafik arayüzlü veya komut istemi tabanlı olabilir. Örneğin Synaptic isimli paket yönetim yazılımı grafik arayüzlü iken, APT komut istemi tabanlı çalışır; yani bir uçbirim aracılığı ile kullanılabilir.
Paket yöneticileri, sadece Linux'a veya UNIX uyarlamalarına özel yazılımlar değildir. Bazı yazılımların, hatta programlama dillerinin bile paket yöneticileri bulunmaktadır. Python dilinde bulunan PIP paket yöneticisi ve Sublime Text adlı metin düzenleyicisinde bulunan "Package Control" (Paket Kontrolü) sistemleri bu duruma örnek verilebilir.
Eskiden bir yazılımı yüklemek için kaynak kodlarını bilgisayarınızda derlemek veya internetten o yazılımı indirmek zorundaydınız. Kaynak kodlarını derlemek zahmetli ve internetten indirmek tehlikeli olunca, paket yöneticileri kendilerine ait paket formatları ile ortaya çıkıp bizleri bu durumdan kurtardı.



Görmüş olduğunuz üzere yazılımın geliştiricisi, yazılımını paketledikten sonra o yazılımı paket deposuna koyuyor. Böylece paketimiz, o depoya bağlı olan bir paket yöneticisi aracılığı ile kontrol edilebiliyor. Örneğin kullanıcımız sistemde güncelleme komutu çalıştırdığı zaman, sistemde halihazırda bulunan paketlerdeki metadata ile paket deposundaki pakette bulunan metadatalar kıyaslanıyor. Bu metadatalar, içerisinde uygulamaya ait sürüm bilgisi, paket ismi gibi temel bilgileri içeriyor.

## Kütüphane ve Bağımlılıklar

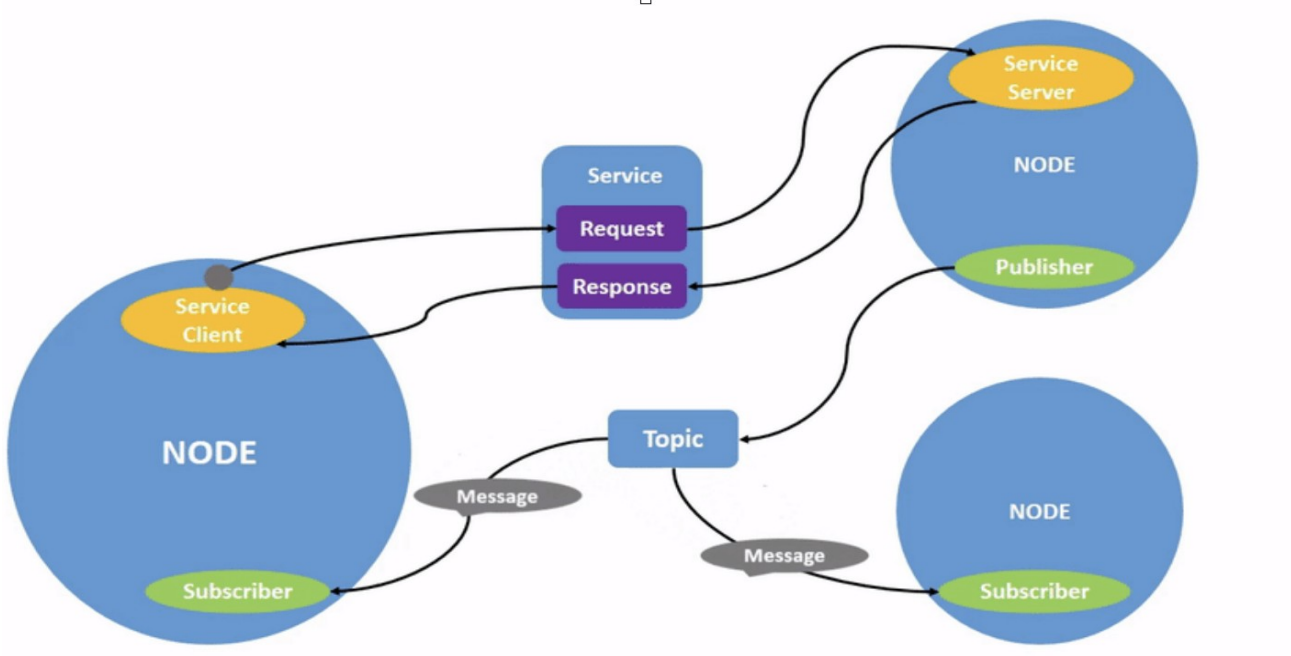
Daha önceden başka yazılımcılar tarafından yazılmış, belirli bir işlevi yerine getiren hazır fonksiyonlar, sınıflar ve kod parçacıklarının topluluğudur.Programcının sıfırdan kodlamakla uğraşmak yerine, hazır bir yapıyı projesine dahil ederek karmaşık problemleri (örneğin matematiksel hesaplamalar, PDF oluşturma, grafik çizme) hızlıca çözmesini sağlar. Örneğin Web arayüzü geliştirmek için kullanılan React veya Python dilinde veri analizi için kullanılan Pandas birer kütüphanedir.

## ROS2

ROS robot kontrol yazılımı geliştirmek için geliştirilmiş açık kaynak bir yazılımdır. İsminde işletim sistemi geçmesine rağmen bir işletim sistemi değildir. Temel olarak

- İletişim altyapısı (yayımla-abone ol tabanlı iletişim ve uzaktan metot çağrısı),
- Çerçeve yazılımları ve araçlar (görselleştirme, yapılandırma altyapısı, kayıt yeniden oynatma) ve
- Ekosistem (simülatörler, algoritmalar, sürücüler ve programlama dili arayüzleri vb.) bütünüdür.

İlk versiyon ROS daha çok akademik projelerde kullanım alanı görüyordu. Ticari projelerde de kullanım sağlamak üzere ROS geliştirilerek şu anki versiyon olan ROS2 yayımlandı.




ROS2 ile gelen en büyük değişiklik iletişim katmanı için DDS ara katmanının seçilmesi oldu. Savunma sanayi projelerinde rüştünü ispat etmiş olan DDS ara katmanı yayımla abone ol mimarisi ile robot bileşenleri arası ve robotlar arası

iletişimi güçlendirmiş oldu. Hali hazırda 3–4 farklı DDS ürünü için ROS2 bağlantısı geliştirmiştir. ROS2 açık kaynak olduğundan ilgili firmalar DDS kütüphanelerini de açık kaynak olarak sektöre sunmuşlardır. ROS 2 aynı zamanda birden fazla robot arası iletişim, gerçek zamanlı iletişim ve farklı platform desteği ile daha geniş bir kullanım alanı buldu. Şu anda insansı robotlardan endüstriyel robotlara ve otonom araçlara kadar farkı alanlarda ROS 2 kullanılmaktadır.

ROS seyir, kontrol, hareket planlama, görü ve simülasyon amaçlarıyla kullanabilmek için açık kaynak olgun kütüphaneleri içermektedir. RVIS isimli 3B görselleştirme aracı ROS ile birlikte kullanılan önemli bir araçtır. Benzer şekilde Gazebo isimli simülasyon aracı da robot geliştiriciler için vazgeçilmez bir araç olarak görülmektedir. Bunun dışında Open CV kütüphanesi ROS 2 de algılama amacı ile kullanılan bir kütüphanedir.

ROS 2 mimarisi temelde gerçek zamanlı dağıtık servis tabanlı mimari üzerinde çalışmaktadır. Robotlarda sensörleri hareket kontrolörleri, algılama algoritmaları, yapay zeka algoritmaları, seyir algoritmaları vb. birer bileşen olarak yer alabilmekte ve bu bileşenler dağıtık ortamda veri alışverişi yaparak robot kontrolünü sağlamaktadır. Veri alışverişi için ROS 2 ile seçilmiş olan DDS ara katmanı bu bileşenlerin birbiri ile dağıtık ortamda haberleşmelerini sağlamaktadır.

ROS 2 DDS ara katmanını doğrudan kullanmak yerine DDS üzerine kendi soyutlama katmanını (rmw) sağlamıştır. Böylelikle DDS ara katmanı ara yüzünün detayları kullanıcıdan soyutlanmıştır. Şu anki ROS 2 sürümlerinde Fast-DDS standart DDS sürümü olarak gelmektedir. Bunun dışında farklı DDS ürünler içinde rmw desteği mevcuttur. Kullanıcı istediği DDS kütüphanesini seçip kullanabilmekte hatta ağ seviyesi birlikte çalışabilirlik sayesinde birden fazla DDS kütüphanesinin aynı projede kullanılması mümkün olmaktadır. Ara katman üzerinde bulunan istemci kütüphanesi yardımı ile geliştirilen uygulamalar ara katman üzerinden haberleşebilmektedirler.

ROS 2 uygulamaları ROS Client Library (RCL) istemci kütüphanesi üzerinden ROS 2 özelliklerine erişmektedir. Rcl kütüphanesi C dili ile yazılmış olup bunun üzerinde C++ dili için Rclcpp ve Pyton dili için Rclpy istemci kütüphaneleri bulunmaktadır. Java, Go gibi diğer diller içinde bağımsız şekilde yazılmış ROS 2 istemci kütüphaneleri bulunmaktadır. İstemci kütüphanesi öncelikle ROS 2 üzerinden konu ve servis yaklaşımlarıyla veri alışverişi sağlamak üzere gerekli standart ara yüz sağlanmaktadır. Ayrıca ROS2 kütüphanesini içinde işletim sistemi soyutlama ve hazır bir takım mikro mimarı yapılar (görev yönetimi, zamanlayıcı vb.) sağlamak üzere kabiliyetler mevcuttur.

Node, topic, mesaj yapısı ve keşif ROS 2 nin temel dağıtık mimarisini oluşturmaktadır. Bu yapı ROS 2 terminolojisinde çizge (graph) olarak adlandırılmaktadır.

*Düğüm (Node):*
Sistem mimarilerinde node (düğüm), karmaşık bir yapıyı oluşturan ve kendine ait belirli bir görevi (veri toplama, veri işleme, karar verme veya eyleme

geçme) bağımsız olarak yürüten her bir modül, bileşen veya işlem birimidir; örneğin şemadaki kamera, zamanlayıcı veya motor sürücü bloklarının her biri birer node olup aralarındaki bağlantılar vasıtasıyla veri alışverişi yaparak sistemin bir bütün halinde çalışmasını sağlar.

*Konu (Topic):*
Genellikle yayınla-abone ol (publish-subscribe) tabanlı haberleşme sistemlerinde, node'lar arasında veri akışının hangi kanal üzerinden yapılacağını belirten etiket veya sanal adres katmanıdır;

Servis (Service):
Bir node'un diğer bir node'dan belirli bir istekte bulunması ve karşılığında senkronize bir yanıt alması gereken durumlar için kullanılan istemci-sunucu (client-server) tabanlı iletişim mekanizmasıdır

Aksiyon (Action):
Servislerden farklı olarak uzun süren, zaman alacak ve süreç boyunca anlık durum (feedback) takibi gerektiren karmaşık görevlerin yürütülmesini sağlayan yapılandırılmış bir iletişim modelidir.

Mesaj (Message):
Node'ların birbiriyle, konular (topics) veya servisler üzerinden haberleşirken paylaştığı en küçük veri birimi veya paketidir

## 2. NVIDIA Jetson ve Ubuntu

### *2.1. NVIDIA Jetson Nedir?*
NVIDIA Jetson, mobil gömülü sistemler pazarına yönelik Ar-Ge çalışmalarında gerçekleştirilen "GPU yoluyla hızlandırılmış paralel işlemlerde", dünyanın lider görsel hesaplama platformudur.
Yüksek performansı ve hesaplamalarda düşük enerji tüketimi ile Jetson,

- İHA(Drones)
- Otonom Robotik Sistemler
- Mobil Medikal Görüntüleme

gibi işlem yoğun alanlarda ve her tip "deep learning" ve görüntüleme uygulamaları için ideal çözümdür.

### *2.2. Yazılım Ortamı*
NVIDIA Jetson yazılım ortamı, yapay zekâ, robotik, görüntü işleme ve otonom sistemler geliştirmek amacıyla kullanılan, NVIDIA tarafından geliştirilen gömülü bilgisayar platformlarına yönelik bir yazılım altyapısıdır. Bu ortamda genellikle Linux tabanlı Ubuntu işletim sistemi kullanılır ve CUDA, TensorRT ile OpenCV gibi araçlardan yararlanılarak görüntü işleme ve yapay zekâ uygulamaları geliştirilir. C++ ve Python gibi programlama dilleriyle desteklenen Jetson platformu, kamera ve sensörlerden alınan verilerin gerçek zamanlı işlenmesini sağlayabilir. Düşük güç tüketimiyle yüksek işlem performansı sunması sayesinde insansız hava araçları (İHA), otonom robotlar ve akıllı görüntüleme sistemleri gibi alanlarda yaygın olarak kullanılır.

### *2.3Otonom Sistemlerde Kullanımı*

NVIDIA Jetson, otonom sistemlerde çevreden alınan verilerin işlenmesi, nesnelerin algılanması ve sistemin uygun kararları verebilmesi için kullanılan bir gömülü bilgisayar platformudur. İnsansız hava araçlarında (İHA), otonom kara araçlarında ve robotlarda kamera, GPS ve diğer sensörlerden gelen veriler Jetson üzerinde işlenerek çevre analizi ve konum değerlendirmesi yapılabilir. Yapay zekâ modelleri, görüntü işleme algoritmaları ve C++ veya Python ile geliştirilen yazılımlar sayesinde nesne tanıma, engel algılama, hedef takibi ve rota planlaması gibi görevler gerçekleştirilebilir. Ayrıca uçuş kontrolcüsü veya diğer donanımlarla haberleşerek otonom sistemin karar verme süreçlerine katkıda bulunabilir. Yüksek işlem gücü ve gerçek zamanlı veri işleme yetenekleri sayesinde NVIDIA Jetson, otonom sistemlerin çevresini algılamasına ve belirlenen görevleri daha bağımsız biçimde yerine getirmesine yardımcı olur.

## 3. Raspberry Pi ve Ubuntu

### *3.1. Raspberry Pi Nedir?*

Üzerinde mikroişlemci, GPIO (General Purpose Input/Output) yani Genel Amaçlı Giriş ve Çıkış pinleri, RAM ve bir bilgisayar için gerekli tüm özelliklere sahip olan tek bir devre kartı (PCB) üzerinde oluşturulmuş bilgisayarlar “Tek Kart Bilgisayar”, İngilizce “Single Board Computer (SBC)” olarak adlandırılır. Günlük hayatta kullandığımız bilgisayarlardan farklı olarak SBC’ler daha az güç tüketirler ve daha küçük boyuta sahiptirler. Genelde ARM mimarisine sahip olan SBC’lerin, x86 işlemciye sahip modelleri de bulunmaktadır. SBC’lerin donanımsal olarak değiştirilemez olması nedeniyle kullanıcılara daha çok güvenlik sunmaktadır. Tek kart bilgisayarlar akıllı ev, otomotiv, sağlık gibi alanlarda kullanılmaktadır.
Raspberry Pi, bünyesinde Linux işletim sistemini barındırmaktadır. Aynı zamanda bir dizi GPIO pini sağlayarak fiziksel yapılacak olan işlemler için elektronik bileşenleri kontrol etmenize ve Internet of Things (IoT) yani Nesnelerin İnterneti’ni keşfetmeye olanak tanır.

### *3.2. Yazılım ortamı*

Raspberry Pi yazılım ortamı, Raspberry Pi adlı tek kartlı bilgisayarlar üzerinde uygulama geliştirmek ve elektronik sistemleri kontrol etmek amacıyla kullanılan yazılım altyapısıdır. Bu cihazlarda genellikle Raspberry Pi OS veya Ubuntu gibi Linux tabanlı işletim sistemleri kullanılır. Python, C ve C++ gibi programlama dilleriyle yazılım geliştirilebilir; ayrıca GPIO pinleri aracılığıyla sensörler, motorlar, LED'ler ve diğer elektronik bileşenler kontrol edilebilir. Visual Studio Code gibi kod editörleri ve terminal araçları, programların yazılması, derlenmesi ve çalıştırılması için kullanılabilir. Raspberry Pi; düşük maliyeti, geniş yazılım desteği ve donanımlarla kolay iletişim kurabilmesi sayesinde robotik, otomasyon, görüntü işleme ve otonom sistem projelerinde yaygın olarak tercih edilir.

### *2.3. Otonom Sistemlerde Kullanımı*

Raspberry Pi, otonom sistemlerde sensörlerden veri toplamak, çevresel koşulları değerlendirmek ve belirli görevleri otomatik olarak gerçekleştirmek amacıyla kullanılan tek kartlı bir bilgisayardır. İnsansız hava araçları (İHA), otonom kara araçları ve robotik projelerde kamera, mesafe sensörleri, GPS

modülleri ve diğer donanımlarla birlikte kullanılabilir. Python ve C++ gibi programlama dilleriyle geliştirilen yazılımlar sayesinde sensör verilerinin işlenmesi, görüntü analizi, engel algılama ve temel karar verme işlemleri gerçekleştirilebilir. GPIO pinleri ve çeşitli haberleşme protokolleri aracılığıyla motorlar, sensörler ve kontrol birimleriyle iletişim kurabilir. Düşük maliyetli ve esnek bir platform olması Raspberry Pi'yi prototip geliştirme, eğitim ve küçük ölçekli otonom sistem projeleri için uygun hâle getirir. Bununla birlikte, yüksek işlem gücü ve gerçek zamanlı performans gerektiren uygulamalarda ek donanımlara veya daha güçlü bilgisayarlara ihtiyaç duyulabilir.

## *4. C++ ve Otonomi Algoritmaları*

### *4.1. C++ Dilinin Otonom Sistemlerdeki Rolü*

C++, yüksek performanslı ve verimli yazılım geliştirmeye olanak sağlayan bir programlama dilidir. Otonom sistemlerde sensör verilerinin işlenmesi, karar verme mekanizmalarının oluşturulması, hareket kontrolü ve gerçek zamanlı işlemlerin yürütülmesi gibi görevlerde önemli bir rol oynar. Bellek ve işlem kaynaklarının verimli kullanılmasına imkân tanıması, C++ dilini insansız hava araçları (İHA), otonom kara araçları ve robotik sistemler için uygun hâle getirir. Bu sistemlerde C++ kullanılarak engel algılama, rota planlama, konum ve yönelim hesaplama, motor kontrolü ve sensörlerden gelen verilerin değerlendirilmesi gibi algoritmalar geliştirilebilir. Ayrıca ROS (Robot Operating System) gibi robotik yazılım altyapılarıyla uyumlu olması sayesinde farklı donanımların ve yazılım bileşenlerinin birlikte çalışmasını sağlar. Özellikle düşük gecikme ve yüksek işlem performansı gerektiren uygulamalarda C++, otonom sistemlerin hızlı, güvenilir ve kararlı bir şekilde çalışmasına katkıda bulunur.

### *4.2. Temel Otonomi Algoritmaları*

Temel otonomi algoritmaları, insansız hava araçları (İHA), otonom kara araçları ve robotların çevrelerinden aldıkları verileri değerlendirerek belirli görevleri bağımsız bir şekilde gerçekleştirmelerini sağlayan matematiksel ve yazılımsal yöntemlerdir. Bu algoritmalar, sistemin çevresini algılamasına, mevcut durumunu değerlendirmesine, karar vermesine ve uygun hareketleri gerçekleştirmesine yardımcı olur. Otonom sistemlerde algılama, konum belirleme, rota planlama, kontrol ve karar verme gibi farklı işlevleri yerine getiren çeşitli algoritmalar kullanılır. Bu algoritmaların birbiriyle uyumlu çalışması, sistemin güvenilir ve verimli bir şekilde görev yapabilmesi açısından önemlidir. C++ gibi programlama dilleri kullanılarak geliştirilen otonomi algoritmaları, sensörlerden ve diğer donanımlardan elde edilen verilerin işlenerek gerçek zamanlı kararlara dönüştürülmesinde önemli bir rol oynar.

#### Durum Makineleri (State Machine)

Durum makinesi, bir otonom sistemin hangi durumda olduğunu takip etmesini ve belirli koşullar gerçekleştiğinde bir durumdan diğerine geçmesini sağlayan bir yazılım modelidir. İnsansız hava araçlarında (İHA) bekleme, kalkış, görev yürütme, eve dönüş ve iniş gibi farklı aşamaların yönetilmesinde kullanılır. Her durumun kendine ait koşulları ve gerçekleştirdiği işlemler bulunur. Örneğin, kalkış komutu verildiğinde sistem kalkış durumuna geçebilir; belirlenen irtifaya ulaşıldığında ise görev yürütme durumuna geçebilir. Beklenmeyen bir durum oluştuğunda sistemin güvenli bir moda geçmesi de durum makinesiyle yönetilebilir. C++ dilinde enum, switch-case ve koşul ifadeleri kullanılarak

temel bir durum makinesi oluşturulabilir. Bu yaklaşım, karmaşık görevlerin daha düzenli, anlaşılır ve kontrol edilebilir bir şekilde yürütülmesini sağlar.

## Yol Planlama (Path Planning)




Yol planlama, otonom bir aracın başlangıç konumundan hedef konumuna ulaşmak için izleyeceği uygun yolu belirleme işlemidir. Bu süreçte hedefin konumu, çevredeki engeller, aracın hareket kabiliyeti ve görev koşulları dikkate alınabilir. A* (A-Star) ve Dijkstra, özellikle harita üzerinde uygun bir yol bulmak için kullanılan temel yol planlama algoritmalarıdır. A* algoritması, hedefe kalan tahmini mesafeyi de hesaba katarak aramayı yönlendirirken Dijkstra algoritması, başlangıç noktasından diğer noktalara olan en düşük maliyetli yolları belirler. İHA'larda yol planlama, belirlenen ara noktalardan geçme ve engellerden uzak bir rota oluşturma gibi görevlerde kullanılabilir. C++ ile geliştirilen yol planlama algoritmaları, harita ve konum verilerini işleyerek sistemin hedefe ulaşmasına yardımcı olur. Ancak hesaplanan yolun gerçek dünyada güvenle izlenebilmesi için aracın hareket ve kontrol sistemleriyle birlikte çalışması gerekir.

## Konum ve Yönelim Kestirimi

Konum ve yönelim kestirimi, otonom sistemin nerede bulunduğunu ve hangi yöne baktığını veya hareket ettiğini belirlemek için farklı sensörlerden gelen verilerin değerlendirilmesi sürecidir. GPS/GNSS alıcıları, ivmeölçerler, jiroskoplar

ve pusulalar bu amaçla kullanılabilen sensörler arasındadır. GPS/GNSS yaklaşık coğrafi konum bilgisi sağlarken ivmeölçerler ivmeyi, jiroskoplar ise açısal hızı ölçer. Sensörlerin tek başına kullanılması çeşitli hata ve belirsizliklere yol açabileceğinden veriler birleştirilerek daha güvenilir bir tahmin elde edilebilir. Bu işlemde Kalman filtresi ve Genişletilmiş Kalman Filtresi (EKF) gibi yöntemlerden yararlanılabilir. İHA'larda konum ve yönelim kestirimi; rotanın takip edilmesi, aracın dengesinin korunması ve uçuş kontrolünün gerçekleştirilmesi açısından önemlidir. C++ ile geliştirilen kestirim yazılımları, sensör verilerini işleyerek uçuş kontrol sistemine gerekli bilgileri sağlayabilir.

### *4.3. ROS 2 ve C++*

ROS 2 ve C++, robotik sistemlerin geliştirilmesinde sıklıkla birlikte kullanılan iki önemli teknolojidir. ROS 2 (Robot Operating System 2), robotların sensörler, motorlar, kameralar ve diğer donanımlarla iletişim kurmasını, farklı yazılım bileşenlerinin birbiriyle veri paylaşmasını sağlayan açık kaynaklı bir robotik yazılım altyapısıdır. C++ ise yüksek performanslı ve hızlı çalışması sayesinde ROS 2 üzerinde otonomi algoritmaları, sensör verisi işleme, hareket kontrolü ve yol planlama gibi işlemlerin geliştirilmesinde kullanılır. Örneğin, otonom bir insansız hava aracında ROS 2 farklı sistemler arasındaki iletişimi yönetirken C++ ile yazılan algoritmalar aracın konum verilerini değerlendirerek hareket kararları alabilir. Bu nedenle ROS 2 ve C++ bilgisi, otonom robotlar ve insansız hava araçları geliştirmek isteyen mühendisler için önemli bir altyapı oluşturur.

## Örnek Uygulama

### *1. Sistemin çalışma mantığı*

### 2. Karar algoritması

Robotun engel mesafesine göre davranışını şu şekilde belirleyebiliriz:

Mesafe 30 cm'den büyükse robot ileri hareket eder.
Mesafe 30 cm veya daha azsa robot durur.
Engel uzaklaştığında robot yeniden hareket eder.

```
if (mesafe > 30) {
    ileri_hareket_et();
} else {
    dur();
}
```

Bu senaryonun basit **sözde kodu** şöyledir:

## 3. ROS 2 bu sistemde ne yapar?

| ROS 2 bileşeni | Görevi |
|---|---|
| `sensor_node` | Sensör verisini okur ve yayınlar. |
| `controller_node` | Veriyi alır ve C++ algoritmasını çalıştırır. |
| `cmd_vel` topic'i | Robotun doğrusal ve açısal hız komutlarını taşır. |
| `rclcpp` | C++ programlarının ROS 2 ile çalışmasını sağlayan |

4. Bu uygulamada hangi kavramlar var.
C++: Değişkenler, koşullar, fonksiyonlar ve sınıflar.
ROS 2: Node, topic, publisher, subscriber ve rclcpp.
Otonomi: Sensör verisi işleme ve koşula bağlı karar verme.
Kontrol: Hız komutu üretme ve motorları yönetme.

## 5. Genel Değerlendirme

Raspberry Pi, NVIDIA Jetson, ROS 2 ve C++, robotik sistemlerin ve otonom araçların geliştirilmesinde farklı görevler üstlenen, birbirini tamamlayan teknolojilerdir. Raspberry Pi, düşük maliyetli ve genel amaçlı bir tek kartlı bilgisayar olarak sensörlerden veri alma, donanımları kontrol etme, temel otomasyon işlemlerini yürütme ve küçük ölçekli robotik projeler geliştirme amacıyla kullanılabilir. NVIDIA Jetson ise özellikle yapay zekâ, görüntü işleme ve derin öğrenme uygulamalarına yönelik GPU hızlandırma yetenekleriyle öne çıkar. Bu nedenle kamera görüntülerinden nesne algılama, hedef takibi ve gerçek zamanlı görüntü analizi gibi işlem yoğun görevlerde avantaj sağlar. ROS 2, bu donanımlar üzerinde çalışan yazılım bileşenlerinin birbiriyle haberleşmesini, sensör verilerinin aktarılmasını ve farklı görevlerin koordineli şekilde yürütülmesini sağlayan robotik yazılım altyapısıdır. C++ ise yüksek performans gerektiren sensör işleme, kontrol, yol planlama ve otonomi algoritmalarının geliştirilmesinde kullanılabilir. Örneğin, otonom bir insansız hava aracında Raspberry Pi yardımcı kontrol ve veri toplama görevlerini üstlenebilirken NVIDIA Jetson kamera verilerini işleyerek nesneleri algılayabilir; ROS 2 bu bileşenler arasındaki veri akışını düzenlerken C++ ile geliştirilen algoritmalar aracın çevresini değerlendirmesine ve uygun hareket kararları üretmesine yardımcı olabilir. Ancak donanımların görev dağılımı, projenin ihtiyaçlarına, işlem yüküne ve kullanılan modellere göre belirlenmelidir; Raspberry Pi de uygun yazılımlar ve donanımlarla bazı görüntü işleme görevlerini gerçekleştirebilir. Sonuç olarak, bu teknolojilerin temel özelliklerini ve birbirleriyle ilişkilerini anlamak, otonom sistemlerin yalnızca yazılımdan veya donanımdan oluşmadığını; algılama, veri işleme, karar verme, kontrol ve haberleşme süreçlerinin birlikte çalışmasıyla geliştirildiğini göstermektedir. Bu bilgi birikimi, otonom sistemler alanında daha karmaşık projeler geliştirmek için önemli bir başlangıç noktası oluşturur.

## 9. Kaynakça


https://tr.wikipedia.org/wiki/Ubuntu_(işletim_sistemi)
Ubuntu - Getting Started with ROS 2
Robotik Yazılım Mimarileri: ROS 2 ve Yeni Nesil Çözümler
Robot İşletim Sistemi 2 (ROS 2) Mimarisi (Medium)
Getting Started with ROS 2: ROS 2 Installation
https://tr.wikipedia.org/wiki/Linux

https://www.datamarket.com.tr/sozluk/terminal/
https://medium.com/@ulkuhos/framework-ve-library-nedir-5b9b6ede0cb3
https://eguvenlihayat.com/e-imzalihayat/yazilim-kutuphanesi-nedir-ve-ne-ise-yarar/
https://medium.com/yazılım-mimarileri/robot-işletim-sistemi-2-ros-2-mimarisi-1345ae6c69dc
https://www.tet.com.tr/jetson-modul-developer-kits